

Contents

guide FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness	1
0. 这篇论文到底改变了什么	1
1. 回到 2022 年的语境	2
2. 原论文阅读地图	2
3. 标准 Attention 到底慢在哪里	2
4. FlashAttention 的方法图	3
5. Online Softmax 是关键数学	3
6. 一行 Q 的张量级别例子	4
7. Backward 为什么要 recompute	5
8. IO 复杂度	5
9. 为什么 FLOPs 变多还更快	6
10. Kernel Fusion	6
11. Block-Sparse FlashAttention	6
12. 实验证据链	7
13. 与本仓库代码怎么对上	7
14. 极简代码: online softmax	7
15. AI 学习者最容易卡住的点	8
16. 现代意义	8
17. 闭卷掌握检查	8
18. 用 AI agent 学这篇的正确方式	9

guide FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

原论文: FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

本地原文 PDF: [learning/kernel-engineering/paper/01_flashattention.pdf](#)

作者: Dao et al.

年份: 2022

类型: paper

0. 这篇论文到底改变了什么

FlashAttention 改变的是我们看待 attention 性能瓶颈的方式。过去很多长上下文论文把焦点放在 FLOPs: 怎么把 $N \times N$ attention 近似成更少计算。FlashAttention 说: 真正拖慢现代 GPU 的, 经常不是算不动, 而是 HBM 读写太多。

核心主张:

标准 attention 慢, 不只是因为 $O(N^2)$ FLOPs,
更是因为它把 $S=QK^T$ 和 $P=\text{softmax}(S)$ 这两个 $N \times N$ 矩阵写回 HBM。

FlashAttention 用 tiling + online softmax,
在 SRAM 里按块计算 exact attention,
避免 materialize $N \times N$ attention matrix。

它是 exact attention, 不是近似 attention。输出数学上等价于:

$O = \text{softmax}(Q K^T / \sqrt{d}) V$

区别在实现路径: 标准实现先造完整 S 和 P ; FlashAttention 只在片上 SRAM 里处理小块, 并维护每行 softmax 的运行统计量。

1. 回到 2022 年的语境

Transformer 想要长上下文, attention 的 $N \times N$ 成本会爆。很多方法尝试 sparse、low-rank、kernelized attention, 把计算复杂度降到近线性。但论文指出, 这些方法常常没有带来真实 wall-clock speedup, 因为它们忽略了 memory access overhead。

GPU 有层级内存:

- HBM: 容量大, 慢一些。A100 约 40-80GB, 带宽约 1.5-2.0TB/s。
- On-chip SRAM: 容量很小, 很快。A100 每个 SM 约 192KB, 估计带宽约 19TB/s。

现代 GPU 计算速度增长很快, 很多操作变成 memory-bound。Softmax、dropout、mask、layer norm 这类操作往往不是算术量大, 而是反复读写内存贵。

FlashAttention 的关键品味是 IO-aware:

不要只数 FLOPs。

要数 HBM $\leftrightarrow$ SRAM 之间搬了多少数据。

2. 原论文阅读地图

建议按下面顺序读:

1. Abstract: 抓住 IO-aware、exact attention、tiling、HBM/SRAM。
2. Figure 1: 看为什么不写 $N \times N$ attention matrix 能带来 GPT-2 attention 7.6x speedup。
3. Section 2.1: 看 GPU memory hierarchy 和 compute-bound / memory-bound。
4. Section 2.2: 看标准 attention 如何 materialize S 和 P 。
5. Section 3.1: 看 tiling、online softmax、recomputation。
6. Algorithm 1: 看 $Q/K/V$ blocks、 $m/l/o$ 的更新。
7. Theorem 1/2: 看 correctness、 $O(N)$ extra memory 和 IO complexity。
8. Section 3.3: 看 block-sparse FlashAttention。
9. Section 4: 看 BERT、GPT-2、LRA、Path-X、runtime/memory benchmark。

如果只读一页, 读 Algorithm 0 和 Algorithm 1 的对比。一个写 S/P 到 HBM, 一个不写。

3. 标准 Attention 到底慢在哪里

给定:

$Q: [N, d]$

$K: [N, d]$

$V: [N, d]$

标准 attention:

$S = Q K^T / \sqrt{d}$ # $[N, N]$

$P = \text{softmax}(S)$ # $[N, N]$

$O = P V$ # $[N, d]$

标准实现的 HBM 路径:

```

1. read Q, K
2. compute S
3. write S to HBM
4. read S
5. compute P = softmax(S)
6. write P to HBM
7. read P, V
8. compute O
9. write O to HBM

```

S 和 P 是 $N \times N$ 。当 $N=4096$ 时，每个 head 就有 16M 个元素；训练还要为 backward 保存中间值，dropout/mask 又会进一步增加读写。

所以标准 attention 的显存和 IO 瓶颈不是抽象的“复杂度很高”，而是非常具体：

把 $N \times N$ 的注意力分数和概率矩阵来回写 HBM。

4. FlashAttention 的方法图

HBM:

Q $[N,d]$, K $[N,d]$, V $[N,d]$

for each K/V block:

load K_j, V_j -> SRAM

for each Q block:

load Q_i and running state O_i, m_i, l_i -> SRAM

compute S_{ij} = Q_i K_j^T

update online softmax stats

update O_i

write O_i, m_i, l_i -> HBM

final:

O_i already normalized / or normalized at block finalization

更直观地看:

standard:

Q, K -> S $[N,N]$ -> P $[N,N]$ -> O

write	write
HBM	HBM

flash:

Q_i, K_j, V_j blocks -> SRAM -> update O_i, m_i, l_i

never materialize full S or P

FlashAttention 多次读某些 Q/O blocks，但避免读写巨大的 $N \times N$ 中间矩阵。因为 SRAM 快很多，这个 trade-off 是赚的。

5. Online Softmax 是关键数学

Softmax 的困难在于一行需要看到所有 columns:

```
softmax(x)_i = exp(x_i - max(x)) / sum_j exp(x_j - max(x))
```

如果按块读 x , 怎么知道全局 $\max(x)$ 和 denominator? 答案是维护两个运行量:

```
m = running max
l = running sum of exp scores under current max
```

当来了新 block s_{block} :

```
m_new = max(m, max(s_block))
l_new = exp(m - m_new) * l
        + sum(exp(s_block - m_new))
```

旧的 denominator 要按新 max 重新缩放, 因为 softmax 为了数值稳定一直在减最大值。

输出 numerator 也要一起缩放:

```
o_new =
    exp(m - m_new) * o_old
    + exp(s_block - m_new) @ V_block
```

```
O = o_final / l_final
```

这就是本地 attention_flash 里的三个状态:

```
m = -math.inf
l = 0.0
o = [0.0] * d
```

每看到一个 K/V block, 就更新 $m/l/o$ 。最终 o/l 等于完整 softmax attention 的输出。

6. 一行 Q 的张量级别例子

假设:

```
N = 8
d = 4
block_n = 3
```

对第 i 行 query:

```
Q_i:      [d]
K_block:  [block_n, d]
V_block:  [block_n, d]
s_block:  [block_n]
p_block:  [block_n]
o:        [d]
m:        scalar
l:        scalar
```

块顺序:

```
K/V 0: tokens 0,1,2
K/V 1: tokens 3,4,5
K/V 2: tokens 6,7
```

FlashAttention 对每个 block 只暂时产生 s_{block} , 不生成完整 $[N]$ 分数行, 更不生成 $[N,N]$ 矩阵。

本地代码:

```
s_block = [  
    sum(Q[i][k] * K[j][k] for k in range(d)) * scale  
    for j in range(j_start, j_end)  
]  
m_new = max(m, max(s_block))  
rescale = exp(m - m_new)  
l = l * rescale  
o = [v * rescale for v in o]
```

这段是导读里最该手写的代码。写出来, FlashAttention 就从 “神奇 CUDA kernel” 变成了可以理解的数学算法。

7. Backward 为什么要 recompute

训练时标准 attention 会保存 S 和 P , 方便 backward 算梯度。但这正是 $O(N^2)$ memory 的来源。

FlashAttention 选择保存:

```
O: [N,d]  
m: [N]  
l: [N]
```

Backward 时再按块从 $Q/K/V$ recompute S_block 和 P_block 。这会多做一些 FLOPs, 但避免从 HBM 读写完整 $N \times N$ attention matrix。

这是一种 selective gradient checkpointing, 但和普通 checkpointing 的区别是: 它不是盲目用算力换显存, 而是针对 GPU memory hierarchy 精确地减少 HBM IO, 所以往往反而更快。

8. IO 复杂度

论文给出:

```
standard attention HBM accesses:  
    Theta( $N*d + N^2$ )
```

```
FlashAttention HBM accesses:  
    Theta( $N^2 * d^2 / M$ )
```

M = SRAM size

并且 Theorem 1 说明 Algorithm 1 返回的就是:

```
softmax( $QK^T$ )V
```

同时只需要 $O(N)$ additional memory beyond inputs and output.

直觉:

- 标准 attention 的大项是 N^2 , 来自 S/P 。
- FlashAttention 的大项取决于 SRAM 能装多大的 block。
- SRAM 越大, block 越大, 需要反复扫描的次数越少, HBM access 越少。

论文还给出 lower bound: 对一段 SRAM size 范围, exact attention 不可能在所有 M 上渐近地比这个 HBM access 更好。这说明 FlashAttention 不只是工程 hack, 而是接近 IO 最优的 exact attention 设计。

9. 为什么 FLOPs 变多还更快

FlashAttention backward 会 recompute attention blocks, 因此 FLOPs 可能增加。但 wall-clock 仍更快, 因为:

GPU FLOPs 很便宜;
HBM traffic 很贵;
softmax/dropout/mask 这些操作 memory-bound;
避免 $N \times N$ HBM traffic 的收益大于 recompute 的代价。

这就是 IO-aware 和 FLOP-aware 的差别。一个算法少算一点 FLOPs, 但读写很多 HBM, 可能更慢; 一个算法多算一点, 但数据停留在 SRAM, 可能更快。

10. Kernel Fusion

标准 attention 可能分成多个 kernel:

```
matmul QK
mask
softmax
dropout
matmul PV
```

每个 kernel 都会读写 HBM。FlashAttention 把这些放进一个 CUDA kernel:

```
load Q/K/V block
matmul
mask
softmax update
dropout if needed
matmul with V
write O/state
```

这种融合避免了中间结果反复出入 HBM。论文也指出, 普通 PyTorch/TensorFlow 这类高层接口很难精细控制这种 memory movement, 所以需要底层 CUDA kernel。

11. Block-Sparse FlashAttention

FlashAttention 本身是 exact dense attention。论文还扩展到 block-sparse:

只计算 sparse mask 中非零的 attention blocks。

如果非零 block 比例是 s , HBM 访问的大项会按 s 缩小。论文报告 block-sparse FlashAttention 能比 FlashAttention 再快 2-4x, 并能扩到 64K sequence length。

但要分清楚:

- FlashAttention: exact attention, 结果等价标准 attention。
- Block-sparse FlashAttention: sparse approximate attention, 依赖 sparsity mask。

很多人把这两个混在一起, 会误以为 FlashAttention 是近似算法, 这是错的。

12. 实验证据链

论文的证据不只看 kernel microbenchmark, 还看训练和模型质量:

- Attention compute: GPT-2 attention 上相对 PyTorch attention 最高 7.6x speedup.
- BERT-large: sequence length 512, 相比 MLPerf 1.1 training speed record 有 15% end-to-end wall-clock speedup.
- GPT-2: sequence length 1K, 相比 HuggingFace/Megatron baseline 训练约 3x faster.
- Long Range Arena: sequence length 1K-4K, 约 2.4x faster.
- GPT-2 perplexity: 长上下文带来约 0.7 perplexity improvement.
- Long-document classification: 长上下文带来 6.4 points lift.
- Path-X: sequence length 16K, 达到 better-than-chance, 报告 61.4% accuracy.
- Path-256: sequence length 64K, block-sparse FlashAttention 报告 63.1% accuracy.
- General attention benchmark: 常见 sequence length 128 到 2K 上, FlashAttention up to 3x faster, 并可扩展到 64K.

读实验时要看两条线:

1. Kernel 层: HBM access 减少是否真的变成 runtime speedup.
2. Model 层: 更长 context 是否真的提升任务质量.

FlashAttention 两条都给了证据。

13. 与本仓库代码怎么对上

本地文件:

- src/flash_attention.py: Python 教学版, 展示 naive attention 和 online softmax attention 等价.
- src/triton_style.py: 帮你理解 tile/block/program 的 Triton 风格思维.
- src/fused_mlp.py: 另一个 fusion pattern, 帮助理解 memory-bound 操作为什么要融合.
- src/capstone_attn_speedup.py: capstone 里可以测 attention 加速/内存差异.
- lectures/04-flashattention.md: 课程版把 FA-2 的 loop 写得更接近 GPU kernel.

建议先跑:

```
python learning/kernel-engineering/src/flash_attention.py
```

它会比较:

```
attention_naive(Q,K,V)
attention_flash(Q,K,V, block_n=3)
```

并断言两者数值一致。这个小测试非常重要: 它证明 “分块 + online softmax” 不是近似, 而是 exact。

14. 极简代码: online softmax

```
import math

def update(m, l, o, scores, values):
    m_new = max(m, max(scores))
    old_scale = math.exp(m - m_new) if m != -math.inf else 0.0

    l = l * old_scale
    o = [x * old_scale for x in o]
```

```

for s, v in zip(scores, values):
    p = math.exp(s - m_new)
    l += p
    for k in range(len(o)):
        o[k] += p * v[k]

return m_new, l, o

```

完整处理完所有 blocks 后:

```
output = [x / l for x in o]
```

如果你能解释 $\text{old_scale} = \exp(m - m_{\text{new}})$, 就真正理解了 online softmax 的数值稳定性。

15. AI 学习者最容易卡住的点

1. S 和 P 不存, 不代表不计算 QK^T 。
 - FlashAttention 仍会按块计算 scores, 只是不把完整矩阵写回 HBM。
2. Exact 不等于没有近似误差之外的数值差异。
 - 数学目标相同, 但 floating point 加法顺序不同, 可能有微小数值差异。
3. $O(N^2)$ FLOPs 还在。
 - Dense FlashAttention 没有把计算复杂度变线性; 它减少的是 memory IO 和 activation memory。
4. SRAM 不是显存。
 - SRAM 是片上小而快的存储; HBM 是 GPU 大显存。
5. Backward recompute 不是倒退。
 - 多算一点换少搬很多数据, 实际更快。

16. 现代意义

今天几乎所有高性能 LLM 训练/推理栈都吸收了 FlashAttention 的思想:

- 长上下文训练离不开 memory-efficient exact attention。
- Triton/CUDA kernel engineering 变成 LLM 工程核心能力。
- 后续 FlashAttention-2/3 继续优化并行化、work partition、Hopper 特性和 FP8。
- vLLM、PagedAttention、MLA、Ring Attention 等系统工作也都在继续围绕 memory movement 做文章。

FlashAttention 最重要的教育意义是: LLM 性能工程不是只看算法复杂度, 也不是只看模型结构。真正的瓶颈常常在数据如何穿过硬件层级。

17. 闭卷掌握检查

1. 标准 attention 为什么要 materialize S 和 P?
2. S 和 P 的 shape 是什么? 为什么是 HBM 瓶颈?
3. FlashAttention 是 exact 还是 approximate?
4. HBM 和 SRAM 的容量/带宽差别是什么?
5. Online softmax 里 m 和 l 分别代表什么?
6. 当新 block 的 max 变大时, 为什么旧的 l/o 要乘 $\exp(m_{\text{old}} - m_{\text{new}})$?
7. Backward 为什么可以不保存 P, 而是 recompute?
8. Standard attention 和 FlashAttention 的 HBM access 复杂度分别是什么?

9. 为什么 FLOPs 更多也可能更快?
10. Block-sparse FlashAttention 和 dense FlashAttention 有什么区别?
11. 论文有哪些 model-level 证据说明长上下文带来质量提升?
12. 本地 attention_flash 为什么能和 attention_naive 输出一致?

18. 用 AI agent 学这篇的正确方式

不要让 agent 只说 “FlashAttention 用 tiling 加速 attention”。更好的 prompt 是:

我正在读 FlashAttention。请你先让我写出标准 attention 的 S/P/O shape。

然后画出哪些矩阵会被写入 HBM。

接着用 $N=8, d=4, \text{block_n}=3$ 带我手算一行 Q 的 online softmax 更新。

每一步都问我 m, l, o 的 shape 和含义。

最后让我解释为什么多做 recompute 反而更快, 以及为什么 dense FlashAttention 是 exact attention。

真正掌握这篇论文的标志是: 你能从 HBM/SRAM 层级解释 attention 为什么慢, 能手写 online softmax 的 m/l/o 更新, 能说明 $N \times N$ 矩阵没有 materialize 但 attention 仍然 exact, 也能把 kernel speedup 和长上下文模型质量提升连接起来。